

**CONCEITOS-
-CHAVE**

depuração.....	421
grupo de teste independente	403
revisão de configuração.....	417
teste alfa.....	417
teste beta.....	417
teste de classe ...	415
teste de disponibilização... ..	420
teste de integração	409
teste de regressão.....	416
teste de sistema ..	418
teste de validação	416
teste unitário	407
V&V.....	402

A estratégia de teste de software fornece um roteiro que descreve os passos a ser executados como parte do teste, define quando esses passos são planejados e então executados, e quanto trabalho, tempo e recursos serão necessários. Portanto, qualquer estratégia de teste deve incorporar planejamento dos testes, projeto de casos de teste, execução dos testes, coleta e avaliação dos dados resultantes.

Uma estratégia de teste de software deve ser flexível o bastante para promover uma abordagem de teste personalizada. Ao mesmo tempo, deve ser rígida o bastante para estimular um planejamento razoável e o acompanhamento, à medida que o projeto progride. Shooman [Sho83] discute essas questões:

De muitas formas, o teste é um processo individualista e o número de tipos diferentes de testes varia tanto quanto as diferentes abordagens de desenvolvimento. Por muitos anos, nossa única defesa contra os erros de programação era um projeto cuidadoso e a inteligência do programador. Estamos agora em uma era na qual as modernas técnicas de projeto [e revisões técnicas] estão nos ajudando a reduzir a quantidade de erros iniciais inerentes ao código. De maneira semelhante, diferentes métodos de teste estão começando a se agrupar em várias abordagens e filosofias distintas.

Essas “abordagens e filosofias” são chamadas de *estratégia* — o tópico que será apresentado neste capítulo. Nos Capítulos 18 a 20, apresentamos os métodos e técnicas de teste que implementam a estratégia.

PANORAMA

O que é? O software é testado para revelar erros cometidos inadvertidamente quando projetado e construído. Mas como conduzir os testes? Devemos estabelecer um plano formal para nossos testes? Devemos testar o programa como um todo ou executar testes somente em uma parte dele? Devemos refazer os testes quando acrescentamos novos componentes ao sistema? Quando devemos envolver o cliente? Essas e muitas outras questões são respondidas quando desenvolvemos uma estratégia de teste de software.

Quem realiza? Uma estratégia para teste de software é desenvolvida pelo gerente de projeto, pelos engenheiros de software e pelos especialistas em testes.

Porque é importante? O teste muitas vezes requer mais trabalho de projeto do que qualquer outra ação da engenharia de software. Se for feito casualmente, perde-se tempo, fazem-se esforços desnecessários, e, ainda pior, erros passam sem ser detectados. Portanto, é razoável estabelecer uma estratégia sistemática para teste de software.

Quais são as etapas envolvidas? O teste começa pelo “pequeno” e passa para o “grande”. Ou

seja, os testes iniciais focalizam um único componente ou um pequeno grupo de componentes relacionados e aplicam-se testes para descobrir erros nos dados e na lógica de processamento que foram encapsulados pelo(s) componente(s). Depois de testados, os componentes devem ser integrados até que o sistema completo esteja pronto. Nesse ponto, são executados muitos testes de ordem superior para descobrir erros ao atender aos requisitos do cliente. À medida que os erros forem descobertos, devem ser diagnosticados e corrigidos usando um processo chamado de depuração.

Qual é o artefato? A especificação do teste documenta a abordagem da equipe de software para o teste, definindo um plano que descreve uma estratégia global e um procedimento designando etapas específicas de teste e os tipos de testes que serão feitos.

Como garantir que o trabalho foi feito corretamente? Revisando a especificação do teste antes do teste, pode-se avaliar a integralidade dos casos de testes e das tarefas de teste. Um plano e um procedimento de teste eficaz levarão a uma construção ordenada do software e à descoberta de erros em cada estágio do processo de construção.

17.1 UMA ABORDAGEM ESTRATÉGICA DO TESTE DE SOFTWARE

Teste é um conjunto de atividades que podem ser planejadas com antecedência e executadas sistematicamente. Por essa razão, deverá ser definido para o processo de software um modelo (*template*) para o teste — um conjunto de etapas no qual pode-se colocar técnicas específicas de projeto de caso de teste e métodos de teste.

Muitas estratégias de teste de software já foram propostas na literatura. Todas elas fornecem um modelo para o teste e todas têm as seguintes características genéricas:

- Para executar um teste eficaz, proceder a revisões técnicas eficazes (Capítulo 15). Fazendo isso, muitos erros serão eliminados antes do começo do teste.
- O teste começa no nível de componente e progride em direção à integração do sistema computacional como um todo.
- Diferentes técnicas de teste são apropriadas para diferentes abordagens de engenharia de software e em diferentes pontos no tempo.
- O teste é feito pelo desenvolvedor do software e (para grandes projetos) por um grupo independente de teste.
- O teste e a depuração são atividades diferentes, mas a depuração deve ser associada com alguma estratégia de teste.

WebRef

Recursos úteis para o teste de software podem ser encontrados em www.mtsu.edu/~storm/.

Uma estratégia de teste de software deve acomodar testes de baixo nível, necessários para verificar se um pequeno segmento de código fonte foi implementado corretamente, bem como testes de alto nível, que validam as funções principais do sistema de acordo com os requisitos do cliente. Uma estratégia deverá fornecer diretrizes para o profissional e uma série de metas para o gerente. Devido ao fato de os passos da estratégia de teste ocorrerem no instante em que as pressões pelo prazo começam a aumentar, deve ser possível medir o progresso no desenvolvimento e os problemas devem ser revelados o mais cedo possível.

17.1.1 Verificação e validação

O teste de software é um elemento de um tópico mais amplo, muitas vezes conhecido como verificação e validação (V&V). *Verificação* refere-se ao conjunto de tarefas que garantem que o software implementa corretamente uma função específica. *Validação* refere-se a um conjunto de tarefas que asseguram que o software foi criado e pode ser rastreado segundo os requisitos do cliente. Boehm [Boe81] define de outra maneira:

Verificação: “Estamos criando o produto corretamente?”

Validação: “Estamos criando o produto certo?”

A definição de V&V abrange muitas atividades de garantia da qualidade do software (Capítulo 16).¹

A verificação e a validação incluem uma ampla gama de atividades de SQA (*software quality assurance*, garantia da qualidade de software): revisões técnicas, auditorias de qualidade e configuração, monitoramento de desempenho, simulação, estudo de viabilidade, revisão de documentação, revisão de base de dados, análise de algoritmo, teste de desenvolvimento, teste de usabilidade, teste de qualificação, teste de aceitação e teste de instalação. Embora a aplicação de teste tenha um papel extremamente importante em V&V, muitas outras atividades também são necessárias.

O teste proporciona o último elemento a partir do qual a qualidade pode ser estimada e, mais pragmaticamente, os erros podem ser descobertos. Mas o teste não deve ser visto como uma rede de segurança. Como se costuma dizer, “Você não pode testar qualidade. Se a qualidade não

“O teste é uma parte inevitável de qualquer trabalho responsável para o desenvolvimento de um sistema de software.”

William Howden



Não seja tolo e não considere o teste como uma “rede de segurança” que detectará todos os erros ocorridos devido a práticas deficientes de engenharia de software. Isso não acontecerá. Enfatize a qualidade e a detecção de erro em todo o processo de software.

¹ Deve-se observar que há uma forte divergência de opinião sobre quais tipos de testes constituem “validação”. Algumas pessoas acreditam que *todo* o teste é verificação e que a validação é executada quando os requisitos são examinados e aprovados, e mais tarde pelo usuário, com o sistema já em operação. Outras pessoas consideram o teste de unidade e de integração (Seções 17.3.1 e 17.3.2) como verificação e o teste de ordem superior (Seções 17.6 e 17.7) como validação.

está lá antes de um teste, ela não estará lá quando o teste terminar”. A qualidade é incorporada ao software através do processo de engenharia de software. A aplicação correta de métodos e ferramentas, de revisões técnicas eficazes, e de um sólido gerenciamento e avaliação conduzem todos à qualidade que é confirmada durante o teste.

Miller [Mil77] relaciona teste de software com garantia de qualidade dizendo que “a motivação que está por trás do teste de programas é a confirmação da qualidade de software com métodos que podem ser economicamente e efetivamente aplicados a todos os sistemas, de grande e pequena escala”.

17.1.2 Organizando o teste de software

Para todo projeto de software, há um conflito de interesses inerente que ocorre logo que o teste começa. As pessoas que criaram o software são agora convocadas para testá-lo. Isso parece essencialmente inofensivo; afinal, quem conhece o programa melhor do que os seus próprios desenvolvedores? Infelizmente, esses mesmos desenvolvedores têm interesse em demonstrar que o programa é isento de erros e funciona de acordo com os requisitos do cliente, e também será concluído dentro do prazo e do orçamento previsto. Cada um desses interesses vai contra o teste completo.

De um ponto de vista psicológico, a análise e o projeto de software (juntamente com a sua codificação) são tarefas construtivas. O engenheiro de software analisa, modela e então cria um programa de computador e sua documentação. Como qualquer outro construtor, o engenheiro de software tem orgulho do edifício que ele construiu e encara com desconfiança qualquer um que tenta estragar sua obra. Quando o teste começa, há uma tentativa sutil, embora definida de “quebrar” aquela coisa que o engenheiro de software construiu. Do ponto de vista do construtor, o teste pode ser considerado como (psicologicamente) destrutivo. Assim, o construtor vai, calmamente, projetando e executando testes que demonstram que o programa funciona, em vez de descobrir os erros. Infelizmente, os erros estão presentes. E se o engenheiro de software não os encontrar, o cliente encontrará!

Frequentemente, há muitas noções incorretas que podem ser inferidas erroneamente a partir da discussão apresentada: (1) que o desenvolvedor de software não deve fazer nenhum teste, (2) que o software deve ser “atirado aos leões”, ou seja, entregue a estranhos que realizarão testes implacáveis, (3) que os testadores se envolvem com o projeto somente no início das etapas do teste. Todas essas afirmativas são incorretas.

O desenvolvedor do software é sempre responsável pelo teste das unidades individuais (componentes) do programa, garantindo que cada uma execute a função ou apresente o comportamento para o qual foi projetada. Em muitos casos, o desenvolvedor faz também o teste de integração — uma etapa de teste que leva à construção (e teste) da arquitetura completa do software. Somente depois que a arquitetura do software foi completada é que o grupo de teste independente se envolve.

O papel de um *grupo independente de teste* (ITG, *independent test group*) é remover problemas inerentes associados com o fato de deixar o criador testar a coisa que ele mesmo criou. O teste independente remove o conflito de interesses que, de outra forma, poderia estar presente. Afinal, o pessoal de ITG é pago para encontrar erros.

No entanto, você não entrega simplesmente o programa para o pessoal do ITG e vai embora. O desenvolvedor e o pessoal do ITG trabalham juntos durante todo o projeto de software para garantir que testes completos sejam realizados. Enquanto o teste está sendo realizado, o desenvolvedor deve estar disponível para corrigir os erros encontrados.

O ITG faz parte da equipe de desenvolvimento de software no sentido de que ele se envolve durante a análise, e o projeto e permanece envolvido (planejando e especificando procedimentos de teste) durante o projeto inteiro. No entanto, em muitos casos o ITG se reporta à organização de garantia de qualidade do software, adquirindo assim um grau de independência que poderia não ser possível se fizesse parte da organização de engenharia de software.

“Otimismo é a doença ocupacional da programação; o teste é seu tratamento.”

Kent Beck

PONTO-CHAVE

Um grupo independente de teste não tem o “conflito de interesses” que os criadores de software podem ter.

“O primeiro erro que as pessoas cometem é pensar que a equipe de teste é responsável por garantir a qualidade.”

Brian Marick

17.1.3 Estratégia de teste de software — a visão ampla

O processo de software pode ser visto como a espiral ilustrada na Figura 17.1. Inicialmente, a engenharia de sistemas define o papel do software e passa à análise dos requisitos de software, onde são estabelecidos o domínio da informação, função, comportamento, desempenho, restrições e critérios de validação para o software. Deslocando-se para o interior da espiral, chega-se ao projeto e finalmente à codificação. Para desenvolver software de computador, percorre-se a espiral para o interior (sentido anti-horário) ao longo de linhas que indicam a diminuição do nível de abstração em cada volta.

Uma estratégia para teste de software pode também ser vista no conceito da espiral (Figura 17.1). O *teste de unidade* começa no centro da espiral e se concentra em cada unidade (por exemplo: componente, classe, ou objeto de conteúdo de WebApp) do software conforme implementado no código-fonte. O teste prossegue movendo-se em direção ao exterior da espiral, passando pelo *teste de integração*, em que o foco está no projeto e construção da arquitetura de software. Continuando na mesma direção da espiral, encontramos o *teste de validação*, em que requisitos estabelecidos como parte dos requisitos de modelagem são validados em relação ao software criado. Finalmente, chegamos ao *teste do sistema*, no qual o software e outros elementos são testados como um todo. Para testar um software de computador, percorre-se a espiral em direção ao seu exterior, no sentido horário, ao longo de linhas que indicam o escopo do teste a cada volta.

Considerando o processo de um ponto de vista procedimental, o teste dentro do contexto de engenharia de software é na realidade uma série de quatro etapas que são implementadas sequencialmente. As etapas estão ilustradas na Figura 17.2. Inicialmente, os testes focalizam cada componente individualmente, garantindo que ele funcione adequadamente como uma unidade, daí o nome *teste de unidade*. O teste de unidade usa intensamente técnicas de teste com caminhos específicos na estrutura de controle de um componente para garantir a cobertura completa e a máxima detecção de erro. Em seguida, o componente deve ser montado ou integrado para formar o pacote completo de software. O *teste de integração* cuida de problemas associados com aspectos duais de verificação e construção de programa. Técnicas de projeto de casos de teste que focalizam em entradas e saídas são mais predominantes durante a integração, embora técnicas que usam caminhos específicos de programa possam ser utilizadas para segurança dos principais caminhos de controle. Depois que o software foi integrado (construído), é executada uma série de *testes de ordem superior*. Os critérios de validação (estabelecidos durante a análise de requisitos) devem ser avaliados. O *teste de validação* proporciona a garantia final de que o software satisfaz a todos os requisitos informativos, funcionais, comportamentais e de desempenho.

A última etapa de teste de ordem superior extrapola os limites da engenharia de software, entrando em um contexto mais amplo de engenharia de sistemas de computadores. O software, uma vez validado, deve ser combinado com outros elementos do sistema (por exemplo, hardware, pessoas, base de dados). O *teste de sistema* verifica se todos os elementos se combinam corretamente e se a função/desempenho global do sistema é conseguida.

? Qual é a estratégia global para teste de software?

WebRef

Recursos úteis para testadores de software podem ser encontrados em www.SQAtester.com.

FIGURA 17.1
Estratégia de teste

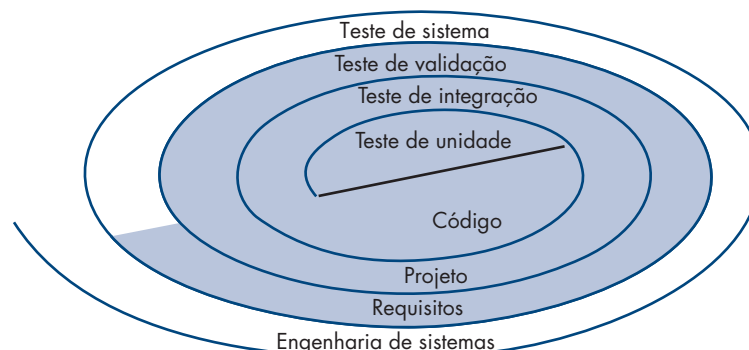
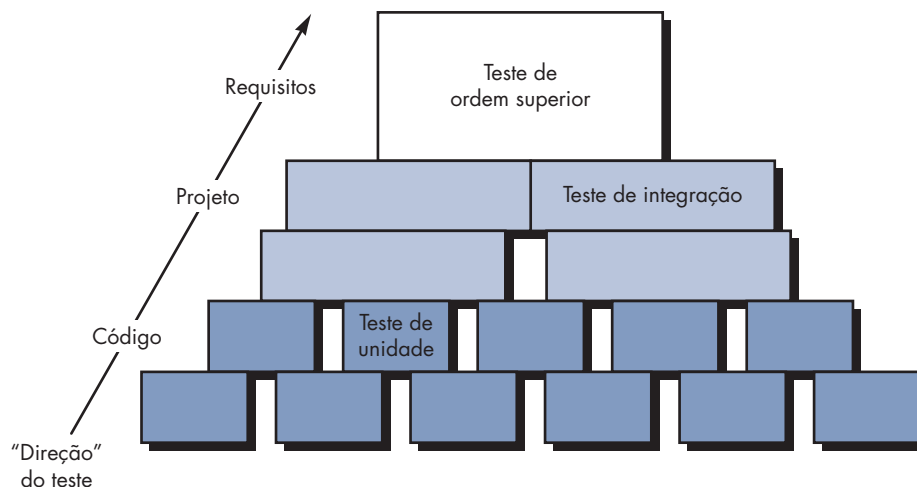


FIGURA 17.2

Etapas de teste de software



CASA SEGURA



Preparando-se para o teste

Cena: No escritório de Doug Miller, quando o projeto no nível de componente está em andamento e começa-se a construção de certos componentes.

Participantes: Doug Miller, gerente de engenharia de software, Vinod, Jamie, Ed e Shakira — membros da equipe de engenharia de software CasaSegura.

Conversa:

Doug: Parece-me que não dedicamos tempo suficiente para falar sobre o teste.

Vinod: É verdade, mas nós estávamos todos um tanto ocupados. Além disso, estivemos pensando sobre isso... Na verdade, mais do que pensando.

Doug (rindo): Eu sei... Todos nós estamos sobrecarregados, mas ainda temos de pensar adiante.

Shakira: Eu gosto da ideia de projetar testes de unidades antes de começar a codificar qualquer um de meus componentes, e é o que estou tentando fazer. Tenho um arquivo grande de testes a ser executados logo que o código dos meus componentes estiver completo.

Doug: Esse é o conceito de Extreme Programming [um processo ágil de desenvolvimento de software; veja o Capítulo 3] não é mesmo?

Ed: É isso mesmo. Apesar de não estarmos usando Extreme Programming diretamente, decidimos que seria uma boa ideia projetar testes unitários antes de criar o componente — o projeto nos dá todas as informações de que precisamos.

Jamie: Eu já fiz a mesma coisa.

Vinod: E eu já assumi o papel de integrador, de forma que, todas as vezes que um dos rapazes passar um componente para mim, eu vou integrá-lo e executar uma série de testes de regressão no programa parcialmente integrado. Estive trabalhando para projetar uma série de testes apropriados para cada função no sistema.

Doug (para Vinod): Com que frequência você fará os testes?

Vinod: Todos os dias... Até que o sistema esteja integrado... Bem, quero dizer, até que o incremento de software que pretendemos fornecer esteja integrado.

Doug: Rapazes, vocês estão mais adiantados do que eu!

Vinod (rindo): Antecipação é tudo no negócio de software, chefe.

17.1.4 Critérios para conclusão do teste

Uma questão clássica surge todas as vezes que se discute teste de software: “Quando podemos dizer que terminamos os testes — como podemos saber que já testamos o suficiente?”. Infelizmente, não há uma resposta definitiva para essa pergunta, mas há algumas respostas pragmáticas e algumas tentativas iniciais e empíricas.

Quando finalizamos o teste?

Uma resposta é: “O teste nunca termina; o encargo simplesmente passa do engenheiro de software para o usuário final”. Todas as vezes que o usuário executa o programa no computador, o programa está sendo testado. Esse fato destaca a importância de outras atividades de garantia de qualidade do software. Outra resposta (um tanto cínica, mas ainda assim, exata) é: “O teste acaba quando o tempo ou o dinheiro acabam”.

Embora alguns profissionais possam argumentar a respeito dessas respostas, o fato é que é necessário um critério mais rigoroso para determinar quando já foram executados testes em número suficiente. A abordagem *engenharia de software de sala limpa* (Capítulo 21) sugere técnicas de uso estatístico [Kel00] que executa uma série de testes derivados de uma amostragem estatística de todas as execuções possíveis do programa por todos os usuários em uma população escolhida. Outros (por exemplo, [Sin99]) defendem o uso de modelagem estatística e teoria de confiabilidade de software para prever a integralidade do teste.

Coletando métricas durante o teste do software e utilizando modelos existentes de confiabilidade de software, é possível desenvolver diretrizes significativas para responder à questão: “Quando terminamos o teste?”. Há uma pequena controvérsia de que ainda resta trabalho a ser feito até que sejam estabelecidas regras quantitativas para o teste, mas as abordagens empíricas que existem no momento são consideravelmente melhores do que a simples intuição.

WebRef

Um glossário claro de termos de teste pode ser encontrado em www.testingstandards.co.uk/living_glossary.htm.

17.2 PROBLEMAS ESTRATÉGICOS

Mais adiante neste capítulo, apresentamos uma estratégia sistemática para teste de software. Mas até mesmo a melhor estratégia fracassará se não for resolvida uma série de problemas e obstáculos. Tom Gilb [Gil95] argumenta que uma estratégia de teste de software terá sucesso quando os testadores de software:

Quais as diretrizes que levam a uma estratégia de teste de software bem-sucedida?

Especifiquem os requisitos do produto de uma maneira quantificável, muito antes de começar o teste. Embora o objetivo principal do teste seja encontrar erros, uma boa estratégia de teste também avalia outras características de qualidade como portabilidade, possibilidade de manutenção e utilidade (Capítulo 14). Esses requisitos devem ser especificados de forma que sejam mensuráveis de tal forma que os resultados do teste não sejam ambíguos.

Definirem explicitamente os objetivos do teste. Os objetivos específicos do teste deverão ser definidos em termos mensuráveis. Por exemplo, a eficiência do teste, a abrangência do teste, o tempo médio entre falhas, o custo para localizar e corrigir defeitos, densidade de defeitos restantes ou frequência de ocorrência e horas de trabalho em testes deverão ser definidos dentro do plano de teste.

Entenderem os usuários do software e desenvolverem um perfil para cada categoria de usuário. Casos de uso que descrevem o cenário de interação para cada classe de usuário podem reduzir o trabalho global de teste, focalizando o teste sobre a utilização real do produto.

Desenvolverem um plano de teste que enfatize o “teste do ciclo rápido”. Gilb [Gil95] recomenda que uma equipe de software “aprenda a testar em ciclos rápidos (2% do trabalho de projeto) de incrementos de funcionalidade e/ou melhora da qualidade úteis ao cliente, ou pelo menos passíveis de experimentação no campo”. O retorno gerado por esses testes de ciclo rápido pode ser usado para controlar os níveis de qualidade e as correspondentes estratégias de teste.

Criarem software “robusto” que seja projetado para testar-se a si próprio. O software deverá ser projetado de maneira a usar técnicas “antidefeitos” (Seção 17.3.1). Isto é, o software deve ser capaz de diagnosticar certas classes de erros. Além disso, o projeto deverá acomodar teste automático e teste de regressão.

Usarem revisões técnicas eficazes como filtro antes do teste. As revisões técnicas (Capítulo 15) podem ser tão eficazes quanto o teste para descobrir erros. Por essa razão, as revisões podem reduzir o trabalho de teste necessário para produzir software de alta qualidade.

WebRef

Uma excelente lista de recursos de teste pode ser encontrada em www.io.com/~wazmo/qa/.

“Teste que leva em conta somente os requisitos do usuário final é como a inspeção de um edifício baseado apenas no trabalho executado pelo projetista de interiores, não considerando as fundações, vigas e tubulações.”

Boris Beizer

Executarem revisões técnicas para avaliar a estratégia de teste e os próprios casos de testes. As revisões técnicas podem descobrir inconsistências, omissões e erros graves na abordagem de teste. Isso poupa tempo e também melhora a qualidade do produto.

Desenvolverem abordagem de melhora contínua para o processo de teste. A estratégia de teste deverá ser medida. As métricas coletadas durante o teste deverão ser usadas como parte de uma abordagem de controle estatístico de processo para o teste de software.

17.3 ESTRATÉGIAS PARA SOFTWARE CONVENCIONAL²

Há muitas estratégias que podem ser utilizadas para testar um software. Em um dos extremos, pode-se esperar até que o sistema esteja totalmente construído e então executar os testes no sistema completo esperando encontrar os erros. Essa abordagem, embora atraente, simplesmente não funciona; resultará em um software defeituoso que desagrade todos os que investiram nele. No outro extremo, você pode executar testes diariamente, sempre que uma parte do sistema seja construída. Essa abordagem, embora menos atraente, pode ser muito eficaz. Infelizmente, alguns desenvolvedores de software hesitam em usá-la. O que fazer?

Uma estratégia de teste que é preferida pela maioria das equipes de software está entre os dois extremos. Ela assume uma visão incremental do teste, começando com o teste das unidades individuais de programa, passando para os testes destinados a facilitar a integração de unidades e culminando com testes que usam o sistema concluído. Cada uma dessas classes de testes é descrita nas próximas seções.

17.3.1 Teste de unidade

O teste de unidade focaliza o esforço de verificação na menor unidade de projeto do software — o componente ou módulo de software. Usando como guia a descrição de projeto no nível de componente, caminhos de controle importantes são testados para descobrir erros dentro dos limites do módulo. A complexidade relativa dos testes e os erros que revelam são limitados pelo escopo restrito estabelecido para o teste de unidade. Esse teste enfoca a lógica interna de processamento e as estruturas de dados dentro dos limites de um componente. Esse tipo de teste pode ser conduzido em paralelo para diversos componentes.



Não é má ideia projetar casos de testes de unidade antes de desenvolver o código para um componente. É bom assegurar-se que você desenvolverá código que passará nos testes.



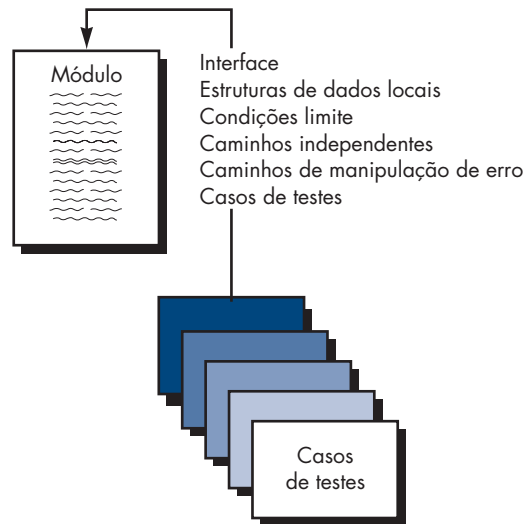
Quais os erros mais comuns encontrados durante o teste de unidade?

Considerações do teste de unidade. Os testes de unidade estão ilustrados esquematicamente na Figura 17.3. A interface do módulo é testada para assegurar que as informações fluam corretamente para dentro e para fora da unidade de programa que está sendo testada. A estrutura de dado local é examinada para garantir que os dados armazenados temporariamente mantenham sua integridade durante todos os passos na execução de um algoritmo. Todos os caminhos independentes da estrutura de controle são usados para assegurar que todas as instruções em um módulo tenham sido executadas pelo menos uma vez. As condições limite são testadas para garantir que o módulo opere adequadamente nas fronteiras estabelecidas para limitar ou restringir o processamento. Finalmente, são testados todos os caminhos de manipulação de erro.

O fluxo de dados por meio da interface de um componente é testado antes de iniciar qualquer outro teste. Se os dados não entram e saem corretamente, todos os outros testes são discutíveis. Além disso, estruturas de dados locais deverão ser ensaiadas e o impacto local sobre dados globais deve ser apurado (se possível) durante o teste de unidade.

O teste seletivo de caminhos de execução é uma tarefa essencial durante o teste de unidade. Casos de testes deverão ser projetados para descobrir erros devido a computações errôneas, comparações incorretas ou fluxo de controle inadequado.

² Neste livro, utilizamos os termos *software convencional* ou *software tradicional* para nos referir às arquiteturas de software do tipo hierárquico comum ou chamada-e-retorno que são frequentemente encontradas em uma variedade de domínios de aplicações. Arquiteturas de software tradicionais *não* são orientadas a objeto e não abrangem WebApps.

FIGURA 17.3**Teste de unidade**

O teste de fronteira é uma das tarefas mais importantes do teste de unidade. O software frequentemente falha nas suas fronteiras. Isto é, os erros frequentemente ocorrem quando o n -ésimo elemento de um conjunto n -dimensional é processado, quando a i -ésima repetição de um laço com i passadas é chamada, quando o valor máximo ou mínimo permitido é encontrado. Casos de testes que utilizam estrutura de dados, fluxo de controle e valores de dados logo abaixo, iguais, ou logo acima dos máximos e mínimos têm grande possibilidade de descobrir erros.

Um bom projeto prevê condições de erro e estabelece caminhos de manipulação de erro para redirecionar ou encerrar ordenadamente o processamento quando ocorre um erro. Yourdon [You75] chama essa abordagem de *antidefeitos*. Infelizmente, há uma tendência de incorporar manipulação de erro no software e nunca testá-lo. Uma história real pode servir como ilustração:

Um sistema de projeto auxiliado por computador (CAD) foi desenvolvido sob contrato. Em um módulo de processamento de transação, um programador brincalhão colocou a seguinte mensagem de manipulação de erro após uma série de testes condicionais que chamavam vários desvios de fluxo de controle: ERRO! VOCÊ NÃO TEM SAÍDA AQUI. Essa “mensagem de erro” foi descoberta por um cliente durante o treinamento com um usuário!

Entre os erros potenciais que deverão ser testados quando a manipulação de erro é avaliada estão: (1) descrição confusa do erro, (2) o erro apontado não corresponde ao erro encontrado, (3) a condição do erro causa intervenção do sistema antes da manipulação do erro, (4) o processamento exceção-condição é incorreto, ou (5) a descrição do erro não fornece informações suficientes para ajudar na localização da causa do erro.

Procedimentos de teste de unidade. O teste de unidade normalmente é considerado um auxiliar para a etapa de codificação. O projeto dos testes de unidade pode ocorrer antes de começar a codificação ou depois que o código-fonte tiver sido gerado. Um exame das informações de projeto fornece instruções para estabelecer casos de testes que provavelmente mostrarão os erros em cada uma das categorias descritas anteriormente. Cada caso de teste deverá ser acooplado com um conjunto de resultados esperados.

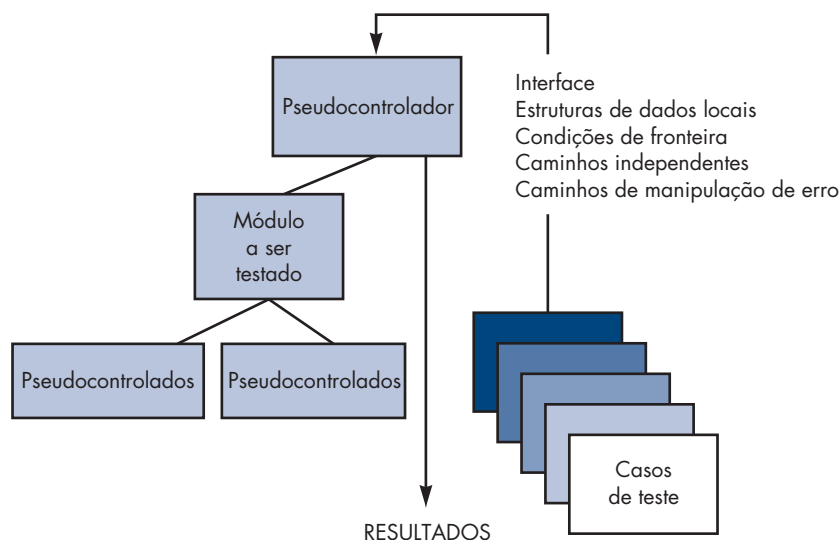
Devido ao fato de um componente não ser um programa independente (*stand-alone*), deve ser desenvolvido um pseudocontrolador (*driver*) e/ou um pseudocontrolado (*stub*) para cada teste de unidade. O ambiente de teste de unidade está ilustrado na Figura 17.4. Em muitas aplicações, um *pseudocontrolador* nada mais é do que um “programa principal” que aceita dados

WebRef

Informações úteis sobre uma ampla variedade de artigos e recursos para “teste ágil” podem ser encontradas em testing.com/agile/.



Projete testes para executar todos os caminhos de manipulação de erro. Se não fizer isso, o caminho pode falhar quando for solicitado, piorando uma situação já ruim.

FIGURA 17.4**Ambiente de teste de unidade**

Há algumas situações nas quais você não terá recursos para fazer um teste de unidade simples. Selecione os módulos críticos ou complexos e faça o teste de unidade apenas neles.

do caso de teste, passa esses dados para o componente (a ser testado), e imprime resultados relevantes. Os *pseudocontroladores* servem para substituir módulos que são subordinados (chamados pelo) componente a ser testado. Um pseudocontrolado, ou “pseudosubprograma”, usa a interface dos módulos subordinados, pode fazer uma manipulação de dados mínima, fornece uma verificação de entrada e retorna o controle para o módulo que está sendo testado.

Pseudocontroladores e pseudocontrolados representam despesas indiretas. Isto é, ambos são softwares que devem ser escritos (projeto formal normalmente não é aplicado), mas que não são fornecidos com o produto de software final. Se os pseudocontroladores e pseudocontrolados são mantidos simples, as despesas reais indiretas são relativamente baixas. Infelizmente, muitos componentes não podem ser adequadamente testados no nível de unidade de modo adequado com software adicional simples. Em tais casos, o teste completo pode ser adiado até a etapa de integração (em que os pseudocontroladores e pseudocontrolados são também usados).

O teste de unidade é simplificado quando um componente com alta coesão é projetado. Quando somente uma função é implementada por um componente, o número de casos de teste é reduzido, e os erros podem ser mais facilmente previstos e descobertos.

17.3.2 Teste de integração

Um novato no mundo do software pode levantar uma questão aparentemente legítima quando todos os módulos tiverem passado pelo teste de unidade: “Se todos funcionam individualmente, porque você duvida que funcionem quando estiverem juntos?”. O problema, naturalmente, é “colocá-los todos juntos” — interfaces. Dados podem ser perdidos através de uma interface; um componente pode ter um efeito inesperado ou adverso sobre outro, subfunções, quando combinadas, podem não produzir a função principal desejada; imprecisão aceitável individualmente pode ser amplificada em níveis não aceitáveis; estruturas de dados globais podem apresentar problemas. Infelizmente, a lista não acaba.

O teste de integração é uma técnica sistemática para construir a arquitetura de software ao mesmo tempo que conduz testes para descobrir erros associados com as interfaces. O objetivo é construir uma estrutura de programa determinada pelo projeto a partir de componentes testados em unidade.

Muitas vezes há uma tendência de tentar integração não incremental; isto é, construir o programa usando uma abordagem big bang. Todos os componentes são combinados com antecedência. O programa inteiro é testado como um todo. E, usualmente, o resultado é o caos!



Adotar a abordagem “big bang” é uma estratégia ineficaz, destinada a fracassar. Integre incrementalmente, testando enquanto trabalha.



Ao desenvolver um cronograma de projeto, você terá de considerar a maneira pela qual a integração ocorrerá, de forma que os componentes estejam disponíveis quando forem necessários.

São encontrados muitos erros. A correção é difícil porque o isolamento das causas é complicado pela vasta expansão do programa inteiro. Uma vez corrigidos esses erros, novos erros aparecem e o processo parece não ter fim.

A integração incremental é o oposto da abordagem big bang. O programa é construído e testado em pequenos incrementos, em que os erros são mais fáceis de isolar e corrigir; as interfaces têm maior probabilidade de ser testadas completamente; e uma abordagem sistemática de teste pode ser aplicada. Nos próximos parágrafos, algumas estratégias de integração incremental diferentes serão discutidas.

Integração descendente. *Teste de integração descendente (top-down)* é uma abordagem incremental para a construção da arquitetura de software. Os módulos são integrados deslocando-se para baixo através da hierarquia de controle, começando com o módulo de controle principal (programa principal). Módulos subordinados ao módulo de controle principal são incorporados à estrutura de uma maneira: primeiro-em-profundidade ou primeiro-em-largura (*depth-first* ou *breadth-first*).

Na Figura 17.5, *integração primeiro-em-profundidade* integra todos os componentes em um caminho de controle principal da estrutura do programa. A seleção de um caminho principal é de certa forma arbitrária e depende das características específicas da aplicação. Por exemplo, selecionando o caminho da esquerda, os componentes M_1 , M_2 , M_5 seriam integrados primeiro. Em seguida, M_8 ou (se necessário para o funcionamento apropriado de M_2) seria integrado M_6 . Depois, são criados os caminhos de controle central e da direita. A *integração primeiro-em-largura* incorpora todos os componentes diretamente subordinados a cada nível, movendo-se através da estrutura horizontalmente. Pela figura, os componentes M_2 , M_3 e M_4 seriam integrados primeiro. Em seguida vem o próximo nível de controle, M_5 , M_6 e assim por diante. O processo de integração é executado em uma série de cinco passos:

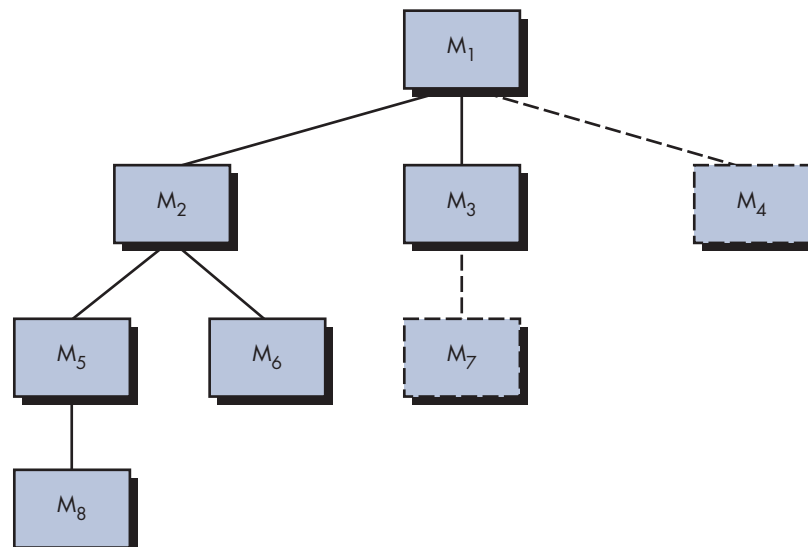


Quais são os passos para a integração descendente?

1. O módulo de controle principal é utilizado como um testador (*test driver*) e todos os componentes diretamente subordinados ao módulo de controle principal são substituídos por pseudocontroladores.
2. Dependendo da abordagem de integração selecionada (isto é, primeiro-em-profundidade ou primeiro-em-largura), pseudocontroladores subordinados são substituídos, um de cada vez, pelos componentes reais.

FIGURA 17.5

Integração descendente



3. Os testes são feitos à medida que cada componente é integrado.
4. Ao fim de cada conjunto de testes, outro pseudocontrolador substitui o componente real.
5. O teste de regressão (discutido mais adiante nesta seção) pode ser executado para garantir que não tenham sido introduzidos novos erros.

O processo continua a partir do passo 2 até que toda a estrutura do programa esteja concluída.

A estratégia de integração descendente verifica os principais pontos de controle ou decisão antecipada no processo de teste. Em uma estrutura de programa bem construída, a tomada de decisão ocorre nos níveis superiores na hierarquia e, portanto, é encontrada primeiro. Se existirem problemas de controle principal, um reconhecimento prévio é essencial. Se for selecionada a integração em profundidade, uma função completa do software pode ser implementada e demonstrada. A demonstração antecipada da capacidade funcional é um gerador de confiança para todos os interessados no programa.

A estratégia descendente parece relativamente descomplicada, mas na prática, podem surgir problemas logísticos. O mais comum desses problemas ocorre quando o processamento em baixos níveis na hierarquia é necessário para testar adequadamente níveis superiores. Pseudocontroladores substituem módulos de baixo nível no início do teste de cima para baixo; portanto, nenhum dado significativo pode fluir ascendentemente na estrutura do programa. Você, como testador, tem três escolhas: (1) adiar muitos testes até que os pseudocontroladores sejam substituídos pelos módulos reais, (2) desenvolver pseudocontroladores com funções limitadas que simulam o módulo real, ou (3) integrar o software de baixo para cima (ascendente).

A primeira abordagem (adiar os testes até que os pseudocontroladores sejam substituídos por módulos reais) pode fazer você perder algum controle sobre a correspondência entre testes específicos e incorporação de módulos específicos. Isso pode resultar em dificuldades para determinar a causa de erros e tende a violar a natureza altamente restrita da abordagem descendente. A segunda abordagem é prática, mas pode levar a uma complicação significativa, à medida que os pseudocontroladores se tornam mais e mais complexos. A terceira abordagem, chamada de integração ascendente, é discutida nos próximos parágrafos.

Integração ascendente. O teste de integração ascendente (*bottom-up*), como o nome diz, começa a construção e o teste com *módulos atômicos* (isto é, componentes nos níveis mais baixos na estrutura do programa). Devido aos componentes serem integrados de baixo para cima, a funcionalidade proporcionada por componentes subordinados a um dado nível está sempre disponível e a necessidade de pseudocontroladores é eliminada. Uma estratégia de integração ascendente pode ser implementada com os seguintes passos:

1. Componentes de baixo nível são combinados em agregados (*clusters*, também chamados de *builds*, construções) que executam uma subfunção específica de software.
2. Um pseudocontrolador (um programa de controle para teste) é escrito para coordenar entrada e saída do caso de teste.
3. O agregado é testado.
4. Os pseudocontroladores (*drivers*) são removidos, e os agregados são combinados movendo-se para cima na estrutura do programa.

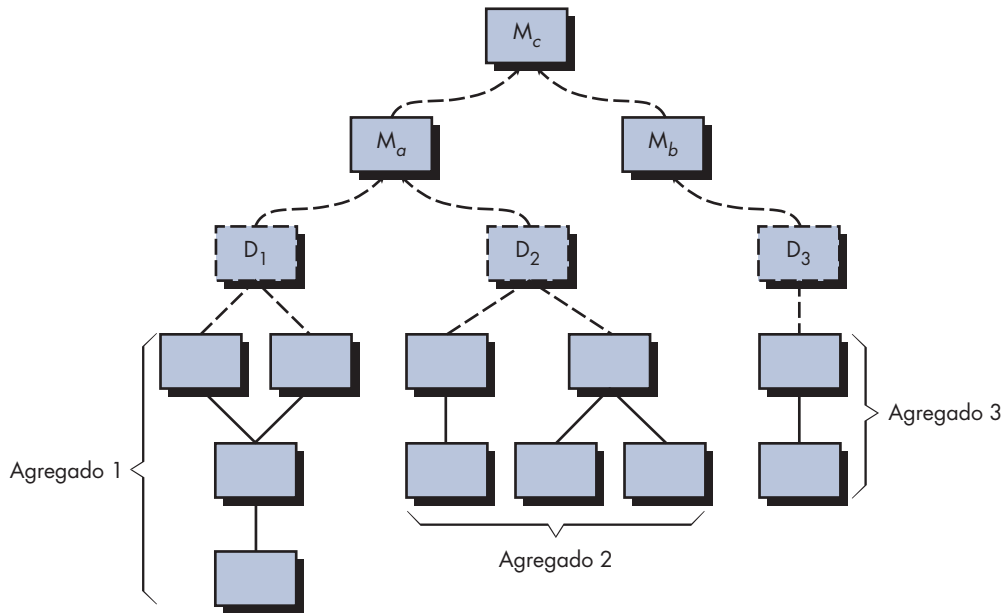
A integração segue o padrão ilustrado na Figura 17.6. Os componentes são combinados para formar os agregados (*clusters*) 1, 2 e 3. Cada um dos agregados é testado usando um pseudocontrolador (mostrado como um bloco tracejado). Componentes nos agregados 1 e 2 são subordinados a M_a . Os pseudocontroladores D_1 e D_2 são removidos e os agregados são interfaceados diretamente a M_a . De forma semelhante, o pseudocontrolador D_3 para o agregado 3 é removido antes da integração com o módulo M_b . M_a e M_b serão ambos integrados finalmente com o componente M_c , e assim por diante.

? Quais problemas podem ser encontrados quando é escolhida a integração descendente?

? Quais são os passos para a integração ascendente?

PONTO-CHAVE

A integração ascendente elimina a necessidade de pseudocontroladores complexos.

FIGURA 17.6**Integração
ascendente**

À medida que a integração se move para cima, a necessidade de pseudocontroladores de testes separados diminui. Na verdade, se os níveis descendentes da estrutura do programa forem integrados de cima para baixo, o número de pseudocontroladores pode ser bastante reduzido e a integração de agregados fica bastante simplificada.



O teste de regressão é uma estratégia importante para reduzir “efeitos colaterais”. Execute testes de regressão toda vez que for feita uma alteração grande no software (incluindo a integração de novos componentes).

Teste de regressão. Cada vez que um novo módulo é acrescentado como parte do teste de integração, o software muda. Novos caminhos de fluxo de dados são estabelecidos, podem ocorrer novas entradas e saídas e nova lógica de controle é chamada. Essas alterações podem causar problemas com funções que antes funcionavam corretamente. No contexto de uma estratégia de teste de integração, o teste de regressão é a reexecução do mesmo subconjunto de testes que já foram executados para assegurar que as alterações não tenham propagado efeitos colaterais indesejados.

Em um contexto mais amplo, testes bem-sucedidos (de qualquer tipo) resultam na descoberta de erros, e os erros devem ser corrigidos. Sempre que o software é corrigido, algum aspecto da configuração do software (o programa, sua documentação, ou os dados que o suportam) é alterado. O teste de regressão ajuda a garantir que as alterações (devido ao teste ou por outras razões) não introduzam comportamento indesejado ou erros adicionais.

O teste de regressão pode ser executado manualmente, reexecutando um subconjunto de todos os casos de teste ou usando ferramentas automáticas de captura/reexecução. Ferramentas de captura/reexecução permitem que o engenheiro de software capture casos de teste e resultados para reexecução e comparação subsequente. O conjunto de teste de regressão (o subconjunto de testes a ser executados) contém três classes diferentes de casos de teste:

- Uma amostra representativa dos testes que usam todas as funções do software.
- Testes adicionais que focalizam as funções de software que podem ser afetadas pela alteração.
- Testes que focalizam os componentes do software que foram alterados.

À medida que o teste de integração progride, o número de testes de regressão pode crescer muito. Portanto, o conjunto de testes de regressão deve ser projetado de forma a incluir somente aqueles testes que tratam de uma ou mais classes de erros em cada uma das funções principais do programa. É impraticável e ineficiente reexecutar todos os testes para todas as funções do programa quando ocorre uma alteração.

PONTO-CHAVE

O teste fumaça pode ser caracterizado como uma estratégia de integração rolante. O software é recriado (com novos componentes acrescentados) e o teste fumaça é realizado todos os dias.

Teste fumaça. *Teste fumaça* é uma abordagem de teste de integração usada frequentemente quando produtos de software são desenvolvidos. É projetado como um mecanismo de marcapasso para projetos com prazo crítico, permitindo que a equipe de software avalie o projeto frequentemente. Em essência, a abordagem teste fumaça abrange as seguintes atividades:

1. Componentes de software que foram traduzidos para um código são integrados em uma “construção” (*build*). Uma construção inclui todos os arquivos de dados, bibliotecas, módulos reutilizáveis e componentes necessários para implementar uma ou mais funções do produto.
2. Uma série de testes é criada para expor erros que impedem a construção de executar corretamente sua função. A finalidade deverá ser descobrir erros “bloqueadores” (*showstopper*) que apresentam a mais alta probabilidade de atrasar o cronograma do software.
3. A construção é integrada com outras construções, e o produto inteiro (em sua forma atual) passa diariamente pelo teste fumaça. A abordagem de integração pode ser descendente ou ascendente.

“Trate a construção diária (*daily build*) como o marcapasso do projeto. Se não houver marcapasso, o projeto está morto.”

Jim McCarthy

A frequência diária de teste do produto inteiro pode surpreender alguns leitores. No entanto, os testes frequentes dão, tanto aos gerentes quanto aos profissionais, uma ideia realística do progresso do teste de integração. McConnell [McC96] descreve o teste fumaça da seguinte maneira:

O teste fumaça deve usar o sistema inteiro de ponta a ponta. Ele não precisa ser exaustivo, mas deve ser capaz de expor os principais problemas. O teste fumaça deve ser bastante rigoroso de forma que, se a construção passar, você pode assumir que ele é estável o suficiente para ser testado mais rigorosamente.

O teste fumaça proporciona muitos benefícios quando aplicado a projetos de engenharia de software complexo e de prazo crítico:

- *O risco da integração é minimizado.* Devido aos teste fumaça serem feitos diariamente, as incompatibilidades e outros erros de bloqueio são descobertos logo, reduzindo assim a probabilidade de impacto sério no cronograma quando os erros são descobertos.
- *A qualidade do produto final é melhorada.* Devido ao fato de a abordagem ser orientada para a construção (integração), o teste fumaça pode descobrir erros funcionais, bem como erros de arquitetura e de projeto no nível de componente. Se esses erros forem corrigidos logo, resultará em melhor qualidade do produto.
- *O diagnóstico e a correção dos erros são simplificados.* Como todas as abordagens de teste de integração, os erros descobertos durante o teste fumaça provavelmente estarão associados com os “novos incrementos do software” — ou seja, o software que acaba de ser acrescentado à(s) construção(ões) é uma causa provável de um erro que acaba de ser descoberto.
- *É mais fácil avaliar o progresso.* A cada dia que passa, uma parte maior do software já está integrada e é demonstrado que funciona. Isso melhora o moral da equipe e dá aos gerentes uma boa indicação de que houve progressos.

Quais benefícios é possível obter do teste fumaça?

WebRef

Indicações para comentários sobre estratégias de teste podem ser encontradas em www.qalinks.com.

Opções estratégicas. Tem havido muita discussão (por exemplo, [Bei84]) sobre as vantagens e desvantagens relativas do teste de integração descendente *versus* ascendente. Em geral, as vantagens de uma estratégia tendem a resultar em desvantagens para outra. A maior desvantagem da abordagem descendente é a necessidade dos pseudocontroladores (*stubs*) e as dificuldades de teste que podem ser associadas com eles. Problemas associados com pseudocontroladores podem ser compensados pela vantagem de se testar antecipadamente as funções principais de controle. A maior desvantagem da integração ascendente é que “o programa como uma entidade não existe enquanto não for acrescentado o último módulo” [Mye79]. Essa desvantagem é compensada por um projeto de casos de teste mais simples e pela ausência de pseudocontroladores.

 O que é um “módulo crítico” e por que devemos identificá-lo?

A escolha de uma estratégia de integração depende das características do software e, algumas vezes, do cronograma do projeto. Em geral, uma abordagem combinada (também chamada de *teste sanduíche*), que usa testes descendentes para níveis superiores da estrutura do programa, acoplados com testes ascendentes para níveis subordinados, pode ser o melhor compromisso.

À medida que o teste de integração é conduzido, o testador deve identificar os módulos críticos. Um *módulo crítico* tem uma ou mais das seguintes características: (1) aborda vários requisitos de software, (2) tem um alto nível de controle (reside em posição relativamente alta na estrutura do programa), (3) é complexo ou sujeito a erros, ou (4) tem requisitos de desempenho bem definidos. Os módulos críticos devem ser testados o mais cedo possível. Além disso, os testes de regressão devem focalizar a função de módulo crítico.

Artefatos do teste de integração. Um plano global para integração do software e uma descrição dos testes específicos são documentados em uma *Especificação de Teste*. Esse produto incorpora um plano de teste e um documento de teste e torna-se parte da configuração do software. O teste é dividido em fases e construções que tratam de características funcionais e comportamentais específicas do software. Por exemplo, o teste de integração para o sistema de segurança *CasaSegura* pode ser dividido nas seguintes fases de teste:

- *Interação com o usuário* (entrada e saída de comandos, representação da tela, processamento e representação de erros)
- *Processamento do sensor* (aquisição da saída do sensor, determinação das condições do sensor, ações necessárias como consequência das condições)
- *Funções de comunicação* (habilidade para se comunicar com a estação de monitoramento central)
- *Processamento do alarme* (testes de ações do software que ocorrem quando um alarme é encontrado)

Cada uma dessas fases de teste de integração representa uma categoria funcional ampla dentro do software e geralmente pode ser relacionada a um domínio específico dentro da arquitetura do software. Portanto, são criadas construções de programa (grupos de módulos) para corresponder a cada fase. Os critérios e testes correspondentes a seguir são aplicados a todas as fases de teste:

 Qual critério deverá ser utilizado para projetar testes de integração?

Integridade da interface. As interfaces interna e externa são testadas à medida que cada módulo (ou agregado) é incorporado à estrutura.

Validade funcional. São executados testes destinados a descobrir erros funcionais.

Conteúdo de informação. São executados testes para descobrir erros associados com estruturas de dados locais ou globais.

Desempenho. São executados testes destinados a verificar os limites de desempenho estabelecidos durante o projeto do software.

Um cronograma para a integração, o desenvolvimento de software de uso geral e tópicos relacionados são também discutidos como parte do plano de teste. São estabelecidas as datas de início e fim para cada fase e são definidas “janelas de disponibilidade” para módulos submetidos a teste de unidade. Uma breve descrição do software de uso geral (pseudocontroladores e pseudocontrolados) concentra-se nas características que poderiam requerer dedicação especial. Finalmente, são descritos o ambiente e os recursos de teste. Configurações de hardware não usuais, simuladores exóticos e ferramentas ou técnicas especiais de teste são alguns dos muitos tópicos que também podem ser discutidos.

Em seguida, é descrito o procedimento detalhado de teste que é necessário para realizar o plano de teste. São descritas a ordem de integração e os testes correspondentes em cada etapa de integração. É incluída também uma lista de todos os casos de testes (anotados para referência subsequente) e dos resultados esperados.

Um histórico dos resultados reais do teste, problemas ou peculiaridades é registrado em um *Relatório de Teste* que pode ser anexado à *Especificação de Teste*, caso desejado. Essas informações podem ser vitais durante a manutenção do software. São apresentados também referências e apêndices apropriados.

Assim como quaisquer outros elementos de uma configuração de software, o formato da especificação de teste pode ser adaptado às necessidades locais de uma organização de engenharia de software. Porém, é importante notar que uma estratégia de integração (contida em um plano de teste) e detalhes do teste (descritos em um procedimento de teste) são ingredientes essenciais e devem estar presentes.

17.4 ESTRATÉGIAS DE TESTE PARA SOFTWARE ORIENTADO A OBJETO³

De forma simplificada, o objetivo do teste é encontrar o maior número possível de erros com um esforço gerenciável durante um intervalo de tempo realístico. Embora esse objetivo fundamental permaneça inalterado para software orientado a objetos, a natureza do software orientado a objeto muda tanto a estratégia quanto a tática de teste (Capítulo 19).

17.4.1 Teste de unidade no contexto OO

Quando consideramos o software orientado a objeto, o conceito de unidades se modifica. O encapsulamento controla a definição de classes e objetos. Isso significa que cada classe e cada instância de uma classe (objeto) empacotam atributos (dados) e as operações que manipulam esses dados. Uma classe encapsulada é usualmente o foco do teste de unidade. No entanto, operações (métodos) dentro da classe são as menores unidades testáveis. Como uma classe pode conter um conjunto de diferentes operações, e uma operação em particular pode existir como parte de um conjunto de diferentes classes, a tática aplicada a teste de unidade precisa modificar-se.

Não podemos mais testar uma única operação isoladamente (a visão convencional do teste de unidade) mas sim como parte de uma classe. Para ilustrar, considere uma hierarquia de classe na qual uma operação *X* é definida para a superclasse e é herdada por várias subclasses. Cada subclasse usa uma operação *X*, mas é aplicada dentro do contexto dos atributos e operações privadas definidas para a subclasse. Como o contexto no qual a operação *X* é utilizada varia de maneira sutil, torna-se necessário testar a operação *X* no contexto de cada uma das subclasses. Isso significa que testar a operação *X* isoladamente (a abordagem de teste de unidade convencional) é usualmente ineficaz no contexto orientado a objeto.

O teste de classe para software OO é o equivalente ao teste de unidade para software convencional. Diferentemente do teste de unidade do software convencional, que tende a focalizar o detalhe algorítmico de um módulo e os dados que fluem através da interface do módulo, o teste de classe para software OO é controlado pelas operações encapsuladas na classe e pelo estado de comportamento da classe.

17.4.2 Teste de integração no contexto OO

Devido ao software orientado a objeto não ter uma estrutura óbvia de controle hierárquico, as estratégias tradicionais de integração descendente e ascendente (Seção 17.3.2) têm pouco significado. Além disso, integrar operações uma de cada vez em uma classe (a abordagem convencional de integração incremental) frequentemente é impossível devido “às interações diretas e indiretas dos componentes que formam a classe” [Ber93].

Há duas estratégias diferentes para teste de integração de sistemas OO [Bin94b]. A primeira, *teste baseado em sequência de execução (thread-based testing)*, integra o conjunto de classes necessárias para responder a uma entrada ou um evento do sistema. Cada sequência de

PONTO-CHAVE

O teste de classe para OO é análogo ao teste de módulo para software convencional. Não é aconselhável testar operações isoladamente.

³ Os conceitos básicos orientados a objeto são apresentados no Apêndice 2.

PONTO-CHAVE

Uma estratégia importante para integração de software OO é o teste com base em sequência de execução (thread). Sequências de execução (threads) são conjuntos de classes que respondem a uma entrada ou evento. Testes fundamentados no uso focalizam classes que não colaboram intensamente com outras classes.

execução é integrada e testada individualmente. O teste de regressão é aplicado para garantir que não ocorram efeitos colaterais. A segunda abordagem de integração, *teste baseado em uso* (*use-based testing*), inicia a construção do sistema testando aquelas classes (chamadas de *classes independentes*) que usam poucas (ou nenhuma) classes *servidoras*. Depois que as classes independentes são testadas, o próximo nível de classes, chamadas de *classes dependentes*, que usam as classes independentes, são testadas. Essa sequência de camadas de teste de classes dependentes continua até que todo o sistema seja construído.

O uso de pseudocontroladores e pseudocontrolados também muda quando é executado o teste de integração de sistemas OO. Pseudocontroladores podem ser utilizados para testar operações no nível mais baixo e para o teste de grupos inteiros de classes. Um pseudocontrolador também pode ser usado para substituir a interface de usuário de maneira que os testes da funcionalidade do sistema podem ser conduzidos antes da implementação da interface. Pseudocontrolados podem ser usados em situações nas quais é necessária a colaboração entre classes, mas uma ou mais das classes colaboradoras ainda não foi totalmente implementada.

O *teste de agregado* (*cluster*) é uma etapa no teste de integração de software OO. Nesse caso, um agregado de classes colaboradoras (determinado examinando-se os modelos CRC e o modelo objeto-relacionamento) é exercitado projetando casos de teste que tentam descobrir erros nas colaborações.

17.5 ESTRATÉGIAS DE TESTE PARA WEBAPPS

A estratégia para teste de WebApp adota os princípios básicos para todo o teste de software e aplica táticas usadas em sistemas OO. Os seguintes passos resumem essa abordagem:

1. O modelo de conteúdo é revisado para descobrir erros.
2. O modelo de interface é revisado para assegurar que todos os casos podem ser acomodados.
3. O modelo de projeto da WebApp é revisado para descobrir erros de navegação.
4. A interface de usuário é testada para descobrir erros na mecânica de apresentação e/ou navegação.
5. Para cada componente funcional é feito o teste de unidade.
6. É testada a navegação através de toda a arquitetura.
7. A WebApp é implementada em uma variedade de ambientes com configurações diferentes e é testada quanto à compatibilidade com cada configuração.
8. São feitos testes de segurança tentando explorar vulnerabilidades na WebApp ou dentro de seu ambiente.
9. São feitos testes de desempenho.
10. A WebApp é testada por uma população de usuários finais controlados e monitorados. Os resultados da interação desses usuários com o sistema são avaliados quanto a erros de conteúdo e de navegação, problemas de utilização, problemas de compatibilidade e confiabilidade e desempenho da WebApp.

Devido a muitas WebApps evoluírem continuamente, o processo de teste é uma atividade contínua, conduzido pelo pessoal de suporte que usa testes de regressão derivados dos testes de desenvolvimento quando a WebApp foi desenvolvida inicialmente. Os métodos de teste para WebApp são considerados no Capítulo 20.

PONTO-CHAVE

A estratégia global para teste de WebApp pode ser resumida nos 10 passos descritos aqui.

WebRef

Excelentes artigos sobre teste de WebApp podem ser encontrados em www.stickyminds.com/testing.asp.

17.6 TESTE DE VALIDAÇÃO

O teste de validação começa quando termina o teste de integração, quando os componentes individuais já foram exercitados, o software está completamente montado como um pacote e os

**PONTO-
-CHAVE**

Como todas as outras etapas de teste, a validação tenta descobrir erros, mas o foco está no nível de requisitos — em coisas que ficarão imediatamente aparentes para o usuário final.

erros de interface já foram descobertos e corrigidos. No nível de validação ou de sistema, a distinção entre software convencional, software orientado a objeto e WebApps desaparece. O teste focaliza ações visíveis ao usuário e saídas do sistema reconhecíveis pelo usuário.

A validação pode ser definida de várias maneiras, mas uma definição simples (embora rigorosa) é que a validação tem sucesso quando o software funciona de uma maneira que pode ser razoavelmente esperada pelo cliente. Nesse ponto, um desenvolvedor de software veterano pode protestar: “Quem ou o quê é o árbitro para decidir o que são expectativas razoáveis?”. Se foi desenvolvida uma *Especificação de Requisitos de Software*, ela descreve todos os atributos do software visíveis ao usuário e contém uma seção denominada *Critério de Validação* que forma a base para uma abordagem de teste de validação.

17.6.1 Critério de teste de validação

A validação de software é conseguida por meio de uma série de testes que demonstram conformidade com os requisitos. Um plano de teste descreve as classes de testes a ser conduzidas e um procedimento de teste define casos de teste específicos destinados a garantir que todos os requisitos funcionais sejam satisfeitos, todas as características comportamentais sejam obtidas, todo o conteúdo seja preciso e adequadamente apresentado, todos os requisitos de desempenho sejam atendidos, a documentação esteja correta e outros requisitos sejam cumpridos (por exemplo, transportabilidade, compatibilidade, recuperação de erro, manutenibilidade).

Após cada caso de teste de validação ter sido conduzido, existe uma dentre duas possíveis condições: (1) a característica de função ou desempenho está de acordo com a especificação e é aceita ou (2) descobre-se um desvio da especificação e é criada uma lista de deficiências. Desvios ou erros descobertos nesse estágio de um projeto raramente podem ser corrigidos antes do prazo de entrega programado. Frequentemente, é necessário negociar com o cliente para estabelecer um método para resolver as deficiências.

17.6.2 Revisão da configuração

Um elemento importante do processo de validação é a *revisão de configuração*. A finalidade da revisão é garantir que todos os elementos da configuração do software tenham sido adequadamente desenvolvidos, estejam catalogados e tenham os detalhes necessários para amparar as atividades de suporte. A revisão de configuração, também chamada de auditoria, é discutida em mais detalhes no Capítulo 22.

17.6.3 Teste alfa e beta

É praticamente impossível para um desenvolvedor de software prever como o cliente realmente usará um programa. As instruções de uso podem ser mal interpretadas, combinações estranhas de dados podem ser usadas regularmente; resultados que pareciam claros para o testador podem ser confusos para um usuário no campo.

Quando é construído um software personalizado para um cliente, são feitos testes de aceitação para permitir ao cliente validar todos os requisitos. Conduzido pelo usuário final e não por engenheiros de software, um teste de aceitação pode variar de um informal *test drive* até uma série de testes planejados e sistematicamente executados. Na verdade, um teste de aceitação pode ser executado por um período de semanas ou meses, descobrindo assim erros cumulativos que poderiam degradar o sistema ao longo do tempo.

Se um software é desenvolvido como um produto para ser usado por muitos clientes, é impraticável executar testes formais de aceitação para cada cliente. Muitos construtores de software usam um processo chamado de teste alfa e beta para descobrir erros que somente o usuário final parece ser capaz de encontrar.

O *teste alfa* é conduzido na instalação do desenvolvedor por um grupo representativo de usuários finais. O software é usado em um cenário natural com o desenvolvedor “espiando por cima dos ombros” dos usuários, registrando os erros e os problemas de uso. Os testes alfa são conduzidos em um ambiente controlado.

“Com uma boa verificação, todos os erros ficam expostos (por exemplo, com uma base suficientemente ampla de testadores beta e codesenvolvedores, quase todos os problemas serão caracterizados rapidamente e a solução será óbvia para alguém).”

E. Raymond

? Qual é a diferença entre um teste alfa e um teste beta?

O teste *beta* é conduzido nas instalações de um ou mais usuários finais. Diferentemente do teste alfa, o desenvolvedor geralmente não está presente. Portanto, o teste beta é uma aplicação “ao vivo” do software em um ambiente que não pode ser controlado pelo desenvolvedor. O cliente registra todos os problemas (reais ou imaginários) encontrados durante o teste beta e relata esses problemas para o desenvolvedor em intervalos regulares. Como resultado dos problemas relatados durante o teste beta, os engenheiros de software fazem modificações e então preparam a liberação do software para todos os clientes.

Uma variação do teste beta, chamada de *teste de aceitação do cliente*, às vezes é executada quando é fornecido software personalizado a um cliente sob contrato. O cliente executa uma série de testes específicos na tentativa de descobrir erros antes de aceitar o software do desenvolvedor. Em alguns casos (por exemplo, um grande sistema corporativo ou governamental) o teste de aceitação pode ser muito formal e levar vários dias ou mesmo semanas.

CASA SEGURA



Preparando para validação

Cena: Escritório de Doug Miller, onde continua o projeto no nível de componente e a criação de certos componentes.

Participantes: Doug Miller, gerente de engenharia de software, Vinod, Jamie, Ed e Shakira — membros da equipe de engenharia de software do CasaSegura.

Conversa:

Doug: O primeiro incremento estará pronto para validação em... Vejamos, três semanas?

Vinod: Certo. A integração está indo bem. Estamos fazendo o teste fumaça diariamente, encontrando alguns erros, mas nada que não se possa resolver. Até agora, tudo bem.

Doug: Fale sobre a validação.

Shakira: Bem, usaremos como base para nosso projeto de teste todos os casos de uso. Ainda não comecei, mas desenvolveremos testes para todos os casos de uso pelos quais sou responsável.

Ed: A mesma coisa aqui.

Jamie: Eu também, mas temos de juntar as nossas ações para teste de aceitação e também para teste alfa e beta, não?

Doug: Sim. Na verdade, estive pensando que poderíamos contratar alguém para nos ajudar na validação. Tenho verba disponível no orçamento... E teríamos um novo ponto de vista.

Vinod: Eu acho que temos tudo sob controle.

Doug: Estou certo que sim, mas um ITG nos dá uma visão independente sobre o software.

Jamie: Estamos com o tempo apertado aqui, Doug. Não temos tempo suficiente para pajear qualquer um que você trouxer aqui.

Doug: Eu sei, eu sei. Mas se um ITG trabalhar a partir dos requisitos e casos de uso, não será necessário muito acompanhamento.

Vinod: Eu ainda acho que temos tudo sob controle.

Doug: Eu sei, Vinod, mas vou insistir nisso. Vamos marcar uma reunião com o representante do ITG ainda esta semana. Vamos dar o pontapé inicial e ver até onde eles chegam.

Vinod: Ok, talvez eles possam aliviar a carga.

17.7 TESTE DE SISTEMA

“Assim como a morte e os impostos, o teste é tanto desagradável quanto inevitável.”

Ed Yourdon

No início deste livro, chamamos a atenção para o fato de que o software é apenas um elemento de um grande sistema de computador. No final, o software é incorporado com outros elementos do sistema (por exemplo, hardware, pessoas, informações), e é executada uma série de testes de integração de sistema e validação. Esses testes estão fora do escopo do processo de software e não são executados somente por engenheiros de software. No entanto, as etapas executadas durante o projeto de software e o teste podem aumentar muito a probabilidade de uma integração de software bem-sucedida em um sistema maior.

Um problema clássico de teste de sistema é a “procura do culpado”. Isso ocorre quando é descoberto um erro e os desenvolvedores de diversos elementos do sistema começam a acusar um ao outro pelo problema. Em lugar de adotar essa postura sem sentido, você deve se antecipar aos problemas potenciais de interface e (1) criar caminhos de manipulação de erro que testem todas as informações vindas de outros elementos do sistema, (2) executar uma série de

testes que simulem dados incorretos ou outros erros potenciais na interface de software, (3) registrar os resultados dos testes para usar como “evidência” se ocorrer a caça ao culpado, e (4) participar do planejamento e projeto de testes do sistema para assegurar que o software seja testado adequadamente.

Teste de sistema é na realidade uma série de diferentes testes cuja finalidade primária é exercitar totalmente o sistema. Embora cada um dos testes tenha uma finalidade diferente, todos funcionam no sentido de verificar se os elementos do sistema foram integrados adequadamente e executam as funções a eles alocadas. Nas próximas seções, discutiremos os tipos de testes de sistema que são vantajosos para sistemas de software.

17.7.1 Teste de recuperação

Muitos sistemas de computador devem se recuperar de falhas e retomar o processamento em pouco ou nenhum tempo de parada. Em alguns casos, um sistema tem de ser tolerante a falhas; ou seja, falhas no processamento não devem causar a paralisação total do sistema. Em outros casos, uma falha no sistema deve ser corrigida dentro de um determinado período de tempo, caso contrário, poderão ocorrer sérios prejuízos financeiros.

Teste de recuperação é um teste do sistema que força o software a falhar de várias formas e verifica se a recuperação é executada corretamente. Se a recuperação for automática (executada pelo próprio sistema), a reinicialização, os mecanismos de verificação, recuperação de dados e reinício são avaliados quanto à correção. Se a recuperação requer intervenção humana, o tempo médio de reparo (*mean-time-to-repair* - MTTR) é avaliado para determinar se está dentro dos limites aceitáveis.

17.7.2 Teste de segurança

Qualquer sistema de computador que trabalhe com informações sensíveis ou que cause ações que podem inadequadamente prejudicar (ou beneficiar) indivíduos, é um alvo para acesso impróprio ou ilegal. As invasões abrangem uma ampla gama de atividades: hackers que tentam invadir sistemas por diversão, funcionários desgostosos que tentam invadir por vingança, indivíduos desonestos que tentam invadir para obter ganhos pessoais ilícitos.

O *teste de segurança* tenta verificar se os mecanismos de proteção incorporados ao sistema vão de fato protegê-lo contra acesso indevido. Citando Beizer [Bei84]: “A segurança de um sistema deve, naturalmente, ser testada quanto à invulnerabilidade por um ataque frontal — mas deve também ser testada quanto à invulnerabilidade por ataques laterais ou pela retaguarda”.

Durante o teste de segurança, o testador faz o papel do indivíduo que quer invadir o sistema. Vale tudo! O testador pode tentar obter senhas por meios externos; pode atacar o sistema com software personalizado projetado para romper quaisquer defesas que tenham sido criadas; pode sobrecarregar o sistema, o sistema pode assim recusar serviço a outros; pode causar erros no sistema proposadamente, esperando poder invadir durante a recuperação; pode examinar dados que não estão em segurança, tentando encontrar a chave para a entrada no sistema.

Com tempo e recursos suficientes, um bom teste de segurança finalmente conseguirá invadir o sistema. O papel do criador do sistema é tornar o custo da invasão maior do que o valor das informações que poderiam ser obtidas.

17.7.3 Teste por esforço

As etapas anteriores de teste resultaram em uma avaliação completa das funções normais do programa e do desempenho. Os testes por esforço (estresse) servem para colocar os programas em situações anormais. Essencialmente, o testador que executa teste por esforço pergunta: “Até onde podemos forçar o sistema até que ele falhe?”.

O *teste por esforço* usa um sistema de maneira que demande recursos em quantidade, frequência ou volumes anormais. Por exemplo, (1) testes especiais podem gerar dez interrupções por segundo, quando uma ou duas é a média normal, (2) a taxa de entrada de dados pode ser au-

“Se você está tentando encontrar verdadeiros defeitos no sistema e ainda não submeteu o seu software a um verdadeiro teste de esforço, então é hora de começar a fazê-lo.”

Boris Beizer

mentada de uma certa magnitude para determinar como as funções de entrada responderão, (3) são executados casos de teste que requerem o máximo de memória ou outros recursos, (4) são executados casos de teste que podem causar problemas de memória em um sistema operacional virtual, (5) são criados casos de teste que podem causar excessiva procura por dados residentes em disco. Essencialmente, o testador tenta quebrar o programa.

Uma variação do teste de esforço é uma técnica chamada *teste de sensibilidade*. Em algumas situações (as mais comuns ocorrem em algoritmos matemáticos), um intervalo muito pequeno de dados, contido dentro dos limites de dados válidos para um programa, podem causar um processamento extremo e até mesmo errôneo ou uma profunda degradação de desempenho. O teste de sensibilidade tenta descobrir combinações de dados dentro de classes de entrada válidas que podem causar instabilidade ou processamento inadequado.

17.7.4 Teste de desempenho

Para sistemas em tempo real e embutidos, um software que execute a função necessária mas não esteja em conformidade com os requisitos de desempenho é inaceitável. O teste de desempenho é projetado para testar o desempenho em tempo de execução do software dentro do contexto de um sistema integrado. O teste de desempenho é feito em todas as etapas no processo de teste. Até mesmo em nível de unidade, o desempenho de um módulo individual pode ser avaliado durante o teste. No entanto, o verdadeiro desempenho de um sistema só pode ser avaliado depois que todos os elementos do sistema estiverem totalmente integrados.

Os testes de desempenho muitas vezes são acoplados ao teste de esforço e usualmente requerem instrumentação de hardware e software. Isto é, frequentemente é necessário medir a utilização dos recursos (por exemplo, ciclos de processador) de forma precisa. Instrumentação externa pode monitorar intervalos de execução, *log* de eventos (por exemplo, interrupções) à medida que ocorrem, e verificar os estados da máquina regularmente. Monitorando o sistema com instrumentos, o testador pode descobrir situações que levam à degradação e possível falha do sistema.

17.7.5 Teste de disponibilização

Em muitos casos, o software deve operar em uma variedade de plataformas e sob mais de um ambiente de sistema operacional. O *teste de disponibilização*, também chamado de *teste de configuração*,

FERRAMENTAS DO SOFTWARE

Planejamento e Gerenciamento do Teste

Objetivo: Essas ferramentas ajudam a equipe de software no planejamento da estratégia de teste escolhida e a gerenciar o processo de teste enquanto ele é executado.



Mecanismos: as ferramentas nessa categoria cuidam do planejamento do teste, armazenamento do teste, gerenciamento e controle, rastreabilidade dos requisitos, integração, rastreamento de erro e geração de relatório. Os gerentes de projeto usam essas ferramentas para suplementar as ferramentas de cronograma de projeto. Testadores usam essas ferramentas para planejar atividades de teste e para controlar o fluxo de informações à medida que o processo de teste avança.

Ferramentas representativas:⁴

QaTraQ Test Case Management Tool, desenvolvida pela TraQ Software (www.testmanagement.com), “sugere uma abordagem estruturada para o gerenciamento do teste”.

QADirector, desenvolvida pela Compuware Corp. (www.compuware.com/qacenter), proporciona um ponto único de controle para gerenciamento de todas as fases do processo de teste.

TestWorks, desenvolvida pela Software Research, Inc. (www.soft.com/Products/index.html), contém um conjunto totalmente integrado de ferramentas de teste, incluindo ferramentas de gerenciamento de teste e relatórios.

OpensourceTesting.org (www.opensourcetesting.org/testmgt.php) lista uma variedade de ferramentas de gerenciamento e planejamento de teste open-source.

NI TestStand, desenvolvida pela National Instruments Corp. (www.ni.com), permite “desenvolver”, gerenciar, e executar sequências de teste escritas em qualquer linguagem de programação”.

⁴ A citação de ferramentas aqui não representa um endosso, mas sim uma amostragem das ferramentas nessa categoria. Na maioria dos casos, os nomes das ferramentas são marcas registradas pelos seus respectivos desenvolvedores.

exercita o software em cada ambiente no qual ele deve operar. Além disso, o teste de disponibilização examina todos os procedimentos de instalação e software especializado de instalação (por exemplo, os “instaladores”) que serão usados pelos clientes e toda a documentação que será usada para fornecer o software para os usuários finais.

Para citar um exemplo, considere a versão do software *CasaSegura* acessível pela Internet que permite ao cliente monitorar o sistema de segurança a partir de uma localização remota. A WebApp *CasaSegura* deve ser testada usando todos os navegadores para a Internet que podem ser encontrados. Um teste de disponibilização mais completo pode abranger combinações de navegadores com vários sistemas operacionais (por exemplo, Linux, Mac OS, Windows). Devido ao fato de a questão da segurança ser um problema importante, uma série completa de testes de segurança deveria ser integrada com o teste de disponibilização.

17.8 A ARTE DA DEPURAÇÃO

“Descobrimos, para nossa surpresa, que não era tão fácil obter os programas certos como pensávamos. Posso me lembrar do momento exato em que percebi que uma grande parte da minha vida desde então seria dedicada a encontrar os erros dos meus próprios programas.”

Maurice Wilkes
descobre a depuração, 1949



Procure evitar uma terceira consequência: A causa é encontrada, mas a “correção” não resolve o problema ou ainda introduz outro erro.



Porque a depuração é tão difícil?

O teste de software é um processo que pode ser sistematicamente planejado e especificado. Pode-se projetar casos de teste, uma estratégia pode ser definida e os resultados podem ser avaliados de acordo com as expectativas prescritas.

A *depuração* ocorre como consequência de um teste bem-sucedido. Isto é, quando um caso de teste descobre um erro, a depuração é o processo que resulta na remoção do erro. Embora a depuração possa e deva ser um processo ordenado, ela ainda é, em grande parte, uma arte. Um engenheiro de software, avaliando os resultados de um teste é frequentemente confrontado com uma indicação sintomática de um problema de software. Isso significa que a manifestação externa do erro e sua causa interna podem não ter nenhuma relação óbvia uma com a outra. O processo mental mal compreendido que conecta um sintoma a uma causa é a depuração.

17.8.1 O processo de depuração

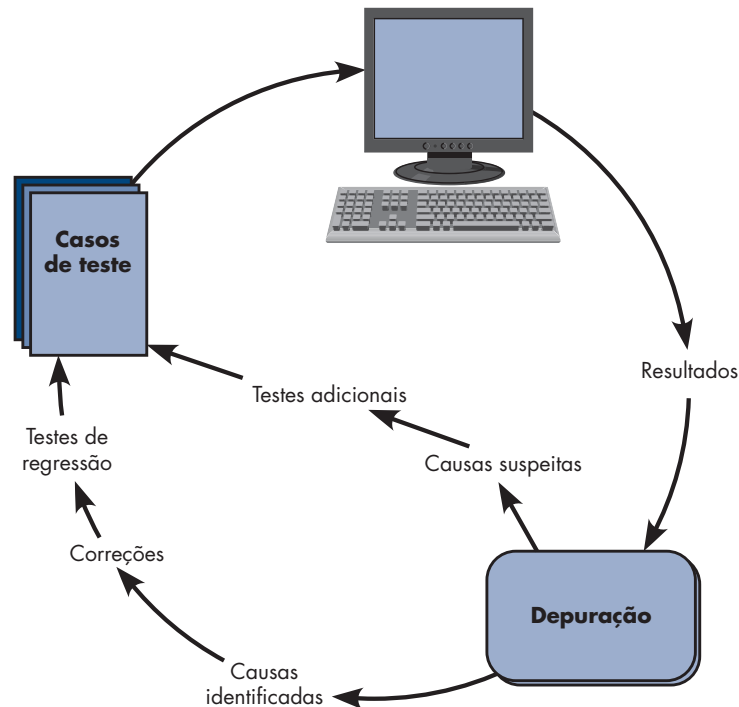
A depuração não é teste, mas frequentemente ocorre em consequência do teste.⁵ De acordo com a Figura 17.7, o processo de depuração começa com a execução de um caso de teste. Os resultados são avaliados e uma falta de correspondência entre o desempenho esperado e o desempenho real é encontrada. Em muitos casos, os dados não correspondentes são um sintoma de uma causa subjacente, embora oculta. O processo de depuração tenta combinar o sintoma com a causa, levando assim à correção do erro.

O processo de depuração usualmente apresentará um dentre dois resultados: (1) a causa será encontrada e corrigida ou (2) a causa não será encontrada. Neste último caso, quem está executando a depuração pode suspeitar de uma causa, criar um caso de teste para ajudar a confirmar aquela suspeita, e trabalhar na correção do erro de uma forma iterativa.

Porque a depuração é tão difícil? Com toda certeza, a psicologia humana (veja a Seção 17.8.2) tem muito mais a ver com a resposta do que a tecnologia de software. No entanto, algumas características dos erros fornecem alguns indícios:

1. O sintoma e a causa podem ser geograficamente remotos. Isto é, o sintoma pode aparecer em uma parte de um programa, enquanto a causa pode realmente estar localizada em um ponto muito afastado. Componentes altamente acoplados (Capítulo 8) pioram essa situação.
2. O sintoma pode desaparecer (temporariamente) quando outro erro for corrigido.
3. O sintoma pode ser, na realidade, causado por não erros (por exemplo, imprecisões de arredondamento).
4. O sintoma pode ser causado por erro humano que não é facilmente rastreável.
5. O sintoma pode ser um resultado de problemas de temporização, e não de problemas de processamento.

⁵ Ao fazer essa afirmativa, estamos assumindo a visão mais ampla possível do teste. Não só o desenvolvedor testa o software antes da entrega, mas também o cliente/usuário testa o software todas as vezes que ele é usado!

FIGURA 17.7**O processo de depuração**

? “Todos sabem que a depuração é duas vezes mais difícil do que escrever um programa a partir de zero. Assim, se você foi o mais esperto possível ao escrever o programa, como poderá depurá-lo?”

Brian Kernighan

6. Pode ser difícil reproduzir com precisão as condições de entrada (por exemplo, uma aplicação em tempo real na qual a ordem das entradas é indeterminada).
7. O sintoma pode ser intermitente. Isso é particularmente comum em sistemas embutidos que acoplam hardware e software inextricavelmente.
8. O sintoma pode ocorrer devido a causas distribuídas por várias tarefas, executando em diferentes processadores.

Durante a depuração, você encontrará erros que variam desde levemente desagradáveis (por exemplo, um formato de saída incorreto) até catastróficos (por exemplo, o sistema falha, causando sérios danos econômicos ou físicos). À medida que as consequências de um erro aumentam, a pressão para encontrar o erro também aumenta. Às vezes, a pressão força alguns desenvolvedores de software a corrigir um erro e ao mesmo tempo introduzir outros dois.

17.8.2 Considerações psicológicas

Infelizmente, parece haver certa evidência de que a destreza na depuração é uma peculiaridade humana inata. Algumas pessoas são boas nisso e outras não. Embora a evidência experimental em depuração seja aberta a muitas interpretações, é possível observar uma grande variação na habilidade de depuração entre programadores com o mesmo nível de formação e experiência. Comentando os aspectos humanos da depuração, Shneiderman [Shn80] afirma:

Depuração é uma das partes mais frustrantes da programação. Ela tem elementos de solução de problemas ou quebra-cabeças, aliados à sensação desagradável de que você cometeu um erro. A ansiedade e indisposição para aceitar a possibilidade de erros aumentam a dificuldade da tarefa. Felizmente, há uma grande sensação de alívio e diminuição da tensão quando o erro finalmente é... Corrigido.

Embora possa ser difícil “aprender” a depurar, várias abordagens do problema podem ser propostas. Vamos examiná-las na Seção 17.8.3.

CASA SEGURA



Depuração

Cena: Cubículo de Ed enquanto é executado o teste de código e unidade.

Participantes: Ed e Shakira — membros da equipe de engenharia de software do CasaSegura.

Conversação:

Shakira (espiando pela entrada do cubículo): Olá... Onde você estava na hora do almoço?

Ed: Aqui mesmo... Trabalhando.

Shakira: Você parece tão mal... O que está havendo?

Ed (suspirando alto): Estive trabalhando neste... Erro desde que o descobri às 9:30 nesta manhã, e já são 2:45... E até agora, nada.

Shakira: Pensei que nós havíamos combinado que não deveríamos gastar mais de uma hora com problemas de depuração sozinhos; depois disso, procurar ajuda, certo?

Ed: Sim, mas...

Shakira (entrando no cubículo): Qual é o problema?

Ed: É complicado, e além disso, eu estive pesquisando por 5 horas. Você não vai poder resolvê-lo em 5 minutos.

Shakira: Desculpe-me... Qual é o problema?

[Ed explica o problema para Shakira, que o examina por 30 segundos sem dizer nada, então...]

Shakira (com um sorriso): Aha!, bem aqui, a variável denominada `setAlarmCondition`. Ela não deveria ser definida como "false" antes de iniciar o laço?

[Ed olha a tela incrédulo, inclina-se para frente e começa a bater com a cabeça no monitor. Shakira, rindo muito, se levanta e sai.]



Defina um limite, digamos, duas horas, para o tempo que você passa tentando depurar um problema por conta própria. Após esse tempo, procure ajuda!

17.8.3 Estratégias de depuração

Independentemente da abordagem adotada, a depuração tem um objetivo primordial — encontrar e corrigir a causa de um erro ou defeito de software. O objetivo é alcançado por uma combinação de avaliação sistemática, intuição e sorte. Bradley [Bra85] descreve a abordagem da depuração da seguinte maneira:

A depuração é uma aplicação direta do método científico que vem sendo desenvolvido há mais de 2.500 anos. A base da depuração é localizar a origem do problema [a causa] por particionamento binário, por meio de hipóteses que preveem novos valores a ser examinados.

Considere um exemplo simples que não é sobre software. Uma lâmpada na sua casa pode não acender. Se nada na casa funciona, a causa pode estar na chave geral da instalação elétrica ou na distribuição na rua; você observa os vizinhos e vê se eles também estão às escuras. Você pega a lâmpada suspeita e a conecta em outro soquete e pega alguma coisa que com certeza está funcionando e conecta naquela tomada suspeita. E assim continua a alternância entre hipótese e teste.

Em geral, foram propostas três estratégias de depuração [Mye79]: (1) força bruta, (2) rastreamento e (3) eliminação da causa. Cada uma dessas estratégias pode ser conduzida manualmente, mas as modernas ferramentas de depuração podem tornar o processo muito mais eficaz.

Táticas de depuração. A categoria *força bruta* para depuração é provavelmente o método mais comum e menos eficiente para isolar a causa de um erro de software. Usamos métodos de depuração do tipo força bruta quando tudo o mais falha. Usando uma filosofia do tipo "deixe o computador encontrar o erro", ocorrem despejos de memória, rastreamentos em tempo de execução e o programa é sobrecarregado com instruções de saída. Espera-se que no meio de toda aquela confusão de informações produzidas consigamos encontrar um indício que possa levar à causa de um erro. Embora a grande quantidade de informações produzidas possa no fim levar ao sucesso, mais frequentemente é uma perda de tempo e trabalho. É preciso pensar primeiro!

Rastreamento (backtracking) é uma abordagem comum de depuração que pode ser usada com sucesso em pequenos programas. Começando no ponto onde o sintoma foi descoberto, o código-fonte é investigado retroativamente (manualmente) até que a causa seja encontrada. Infelizmente, à medida que o número de linhas do código-fonte aumenta, o número de caminhos retroativos potenciais pode se tornar demasiadamente grande.

"O primeiro passo para reparar um programa com problema é fazê-lo falhar repetidamente (no exemplo mais simples possível)."

T. Duff

A terceira abordagem da depuração — *eliminação da causa* — é manifestada por indução ou dedução e introduz o conceito de particionamento binário. Os dados relacionados com a ocorrência do erro são organizados para isolar as causas potenciais. É imaginada uma “hipótese de causa” e os dados mencionados anteriormente são usados para provar ou negar a hipótese. Alternativamente, é preparada uma lista de todas as causas possíveis e são conduzidos os testes para eliminar cada uma delas. Se os testes iniciais indicam que uma determinada causa hipotética promete resultado, os dados são refinados tentando isolar o defeito.

Depuração automática. Cada uma dessas abordagens de depuração pode ser complementada com ferramentas de depuração que podem fornecer suporte semiautomático para o engenheiro de software à medida que são tentadas as estratégias. Hailpern e Santhanam [Hai02] resumem o estado dessas ferramentas quando afirmam “... muitas novas abordagens têm sido propostas e há muitos ambientes comerciais para depuração disponíveis. Ambientes Integrados de Desenvolvimento (IDE) proporcionam uma maneira de capturar alguns dos erros predeterminados específicos de linguagem (por exemplo, falta de caracteres de fim de instrução, variáveis indefinidas, e assim por diante) sem requerer compilação”. Há disponível uma ampla variedade de compiladores de depuração, auxílios dinâmicos de depuração (“rastreadores”), geradores de casos de teste automáticos e ferramentas de mapeamento de referências cruzadas. No entanto, as ferramentas não substituem uma avaliação cuidadosa fundamentada num modelo completo de projeto e um código-fonte claro.

FERRAMENTAS DO SOFTWARE



Depuração

Objetivo: essas ferramentas proporcionam auxílio automático para aqueles que devem depurar problemas de software. A intenção é fornecer informações que podem ser difíceis de obter se o processo de depuração for abordado manualmente.

Mecanismos: muitas ferramentas de depuração são específicas quanto à linguagem de programação e quanto ao ambiente.

Ferramentas representativas:⁶

Borland Gauntlet, distribuída pela Borland (www.borland.com), ajuda no teste e na depuração.

Coverty Prevent SQS, desenvolvida pela Coverty (www.coverty.com), proporciona auxílio de depuração para C++ e Java.

C++Test, desenvolvida pela Parasoft (www.parasoft.com), é uma ferramenta de teste de unidade que suporta uma ampla gama de testes em código C e C++. Características de depuração ajudam no diagnóstico de erros encontrados.

CodeMedic, desenvolvida pela NewPlanet Software (www.newplanetsoftware.com/medic/), proporciona uma interface gráfica para o depurador UNIX padrão, gdb, e implementa suas características mais importantes. O gdb atualmente suporta C/C++, Java, PalmOS, vários sistemas embutidos, linguagem Assembly, FORTRAN e Modula-2.

GNATS, um aplicativo freeware (www.gnu.org/software/gnats/), é um conjunto de ferramentas para rastrear bug reports.

O fator humano. Qualquer discussão sobre abordagens e ferramentas de depuração é incompleta se não mencionar um poderoso aliado — outras pessoas! Um ponto de vista novo, não ofuscado por horas de frustração, pode fazer maravilhas.⁷ Uma máxima final para a depuração seria: “Quando tudo mais falha, procure ajuda!”.

17.8.4 Correção do erro

Uma vez encontrado um defeito, ele precisa ser corrigido. Mas, conforme já observamos, a correção de um defeito pode introduzir outros erros e, portanto, causar mais danos do que

⁶ A citação de ferramentas aqui não representa um endosso, mas sim uma amostragem das ferramentas nesta categoria. Na maioria dos casos, os nomes das ferramentas são marcas registradas pelos seus respectivos desenvolvedores.

⁷ O conceito de programação aos pares (recomendado como parte do modelo Extreme Programming discutido no Capítulo 3) proporciona um mecanismo para “depuração”, à medida que o software é projetado e codificado.

trazer benefícios. Van Vleck [Van89] sugere três perguntas simples que você deve fazer antes de fazer a “correção” que remove a causa de um defeito:

“O melhor testador não é aquele que encontra o maior número de erros... O melhor testador é aquele que consegue corrigir o maior número de erros.”

Cem Kaner et al.

1. *A causa do defeito é reproduzida em alguma outra parte do programa?* Em muitas situações, o defeito de um programa é causado por um padrão incorreto de lógica que pode ser reproduzido em qualquer outro ponto. Uma consideração explícita do padrão lógico pode resultar na descoberta de outros erros.
2. *Qual o “próximo defeito” que pode ser introduzido pelo reparo que estou prestes a fazer?* Antes de fazer a correção, o código-fonte (ou, melhor, o projeto) deverá ser avaliado para verificar acoplamento de lógica e estruturas de dados. Se a correção tem de ser feita em uma parte do programa altamente acoplada, deve ser tomado cuidado especial ao fazer qualquer alteração.
3. *O que poderíamos ter feito para evitar esse defeito já no início?* Essa pergunta é o primeiro passo para o estabelecimento de uma abordagem estatística de garantia de qualidade de software (Capítulo 16). Se você corrige o processo juntamente com o produto, o erro será removido do programa em questão e pode ser eliminado em todos os programas futuros.

17.9 RESUMO

O teste de software absorve a maior porcentagem do esforço técnico em um processo de software. Independentemente do tipo de software criado, uma estratégia para planejamento sistemático de teste, execução e controle começa considerando pequenos elementos do software e se encaminha para fora no sentido de abranger o programa como um todo.

O objetivo do teste de software é descobrir erros. Para o software convencional, esse objetivo é atingido por meio de uma série de passos de teste. Testes de unidade e de integração concentram-se na verificação funcional de um componente e na incorporação dos componentes na arquitetura de software. O teste de validação demonstra os requisitos de rastreabilidade do software, e o teste de sistema aprova o software quando ele é incorporado em um sistema maior. Cada passo do teste é realizado por meio de uma série de técnicas sistemáticas de teste que auxiliam no projeto dos casos de testes. Em cada passo do teste, o nível de abstração com o qual o software é considerado é ampliado.

A estratégia para teste de software OO começa com testes que exercitam as operações dentro de uma classe e depois passa para o teste baseado em sequências de execução para integração. Sequências de execução são conjuntos de classes que respondem a uma entrada ou evento. Testes baseados em uso focalizam classes que não colaboram intensamente com outras classes.

WebApps são testadas de maneira muito semelhante aos sistemas OO. No entanto, os testes são projetados para exercitar conteúdo, funcionalidade, interface, navegação e aspectos de desempenho da WebApp e segurança.

Diferentemente do teste (uma atividade sistemática, planejada), a depuração pode ser vista como uma arte. Começando com a indicação sintomática de um problema, a atividade de depuração deve rastrear a causa de um erro. Dentre os vários recursos disponíveis durante a depuração, o mais valioso é o conselho de outros membros da equipe de engenharia de software.

PROBLEMAS E PONTOS A PONDERAR

17.1. Usando as suas próprias palavras, descreva a diferença entre verificação e validação. Ambas usam métodos de projeto de caso de teste e estratégias de teste?

17.2. Liste alguns dos problemas que podem ser associados com a criação de um grupo de teste independente. Um grupo ITG e um grupo SQA são formados pelas mesmas pessoas?

17.3. É sempre possível desenvolver uma estratégia para teste de software que use a sequência de passos de teste descrita na Seção 17.1.3? Que possíveis complicações podem surgir para sistemas embutidos?

17.4. Porque um módulo altamente acoplado é difícil de testar em unidade?

17.5. O conceito de antidefeito (*antibugging*) (Seção 17.2.1) é uma maneira extremamente eficaz de proporcionar auxílio de depuração interno quando um erro é descoberto:

- a. Desenvolver uma série de diretrizes para antidepuração.
- b. Discutir vantagens do uso da técnica.
- c. Discutir as desvantagens.

17.6. Como o cronograma de projeto pode afetar o teste de integração?

17.7. O teste de unidade é possível ou até mesmo desejável em todas as circunstâncias? Forneça exemplos para justificar a sua resposta.

17.8. Quem deve executar o teste de validação — o desenvolvedor do software ou o usuário do software? Justifique a sua resposta.

17.9. Desenvolva uma estratégia de teste completa para o sistema *CasaSegura* discutido anteriormente neste livro. Documente-a em uma *Especificação de Teste*.

17.10. Como projeto de classe, desenvolva um *Guia de Depuração* para a sua instalação. O guia deve fornecer informações orientadas a linguagem e sistema que tenham sido aprendidas na escola de contratempos! Comece com um esboço dos tópicos que serão revisados pela classe e pelo seu professor. Distribua o guia para os outros no seu ambiente local.

LEITURAS E FONTES DE INFORMAÇÃO COMPLEMENTARES

Praticamente todos os livros sobre teste de software discutem estratégias juntamente com métodos para projeto de caso de teste. Everett e Raymond (*Software Testing*, Wiley-IEEE Computer Society Press, 2007), Black (*Pragmatic Software Testing*, Wiley, 2007), Spiller e colaboradores (*Software Testing Process: Test Management*, Rocky Nook, 2007), Perry (*Effective Methods for Software Testing*, 3d ed., Wiley, 2005), Lewis (*Software Testing and Continuous Quality Improvement*, 2d ed., Auerbach, 2004), Loveland et al. (*Software Testing Techniques*, Charles River Media, 2004), Burnstein (*Practical Software Testing*, Springer, 2003), Dustin (*Effective Software Testing*, Addison-Wesley, 2002), Craig and Kaskiel (*Systematic Software Testing*, Artech House, 2002), Tamres (*Introducing Software Testing*, Addison-Wesley, 2002), Whittaker (*How to Break Software*, Addison-Wesley, 2002) e Kaner et al. (*Lessons Learned in Software Testing*, Wiley, 2001) são apenas alguns dos muitos livros que discutem princípios, conceitos, estratégias e métodos de teste.

Para aqueles leitores que se interessam por métodos ágeis de desenvolvimento de software, Crispin e House (*Testing Extreme Programming*, Addison-Wesley, 2002) e Beck (*Test Driven Development: ByExample*, Addison-Wesley, 2002) apresentam estratégias de teste e táticas para Extreme Programming.

Kaner e colaboradores (*Lessons Learned in Software Testing*, Wiley, 2001) apresentam uma coleção de mais de 300 “lições” pragmáticas (diretrizes) que todo testador de software deveria conhecer.

Watkins (*Testing IT: An Off-the-Shelf Testing Process*, Cambridge University Press, 2001) estabelece uma estrutura de teste eficaz para todos os tipos de software desenvolvido e adquirido. Manges e O'Brien (*Agile Testing with Ruby and Rails*, Apress, 2008) tratam das estratégias de teste e técnicas para a linguagem de programação Ruby e estrutura para a Internet.

Sykes e McGregor (*Practical Guide to Testing Object-Oriented Software*, Addison-Wesley, 2001), Bashir e Goel (*Testing Object-Oriented Software*, Springer-Verlag, 2000), Binder (*Testing Object-Oriented Systems*, Addison-Wesley, 1999), Kung et al. (*Testing Object-Oriented Software*,

IEEE Computer Society Press, 1998), e Marick (*The Craft of Software Testing*, Prentice-Hall, 1997) apresentam estratégias e métodos para teste de sistemas orientados a objeto.

Diretrizes para depuração podem ser encontradas nos livros de Grötter et al. (*The Developer's Guide to Debugging*, Springer, 2008), Agans (*Debugging*, Amacon, 2006), Zeller (*Why Programs Fail: A Guide to Systematic Debugging*, Morgan Kaufmann, 2005), Tells e Hsieh (*The Science of Debugging*, The Coreolis Group, 2001), e Robbins (*Debugging Applications*, Microsoft Press, 2000). Kaspersky (*Hacker Debugging Uncovered*, A-List Publishing, 2005) trata da tecnologia de ferramentas de depuração. Younessi (*Object-Oriented Defect Management of Software*, Prentice-Hall, 2002) apresenta técnicas para cuidar de defeitos encontrados em sistemas orientados a objeto.

Beizer [Bei84] apresenta uma interessante “sistemática de defeitos” que pode levar a métodos eficazes para planejamento de teste.

Livros dos autores Madisetti e Akgul (*Debugging Embedded Systems*, Springer, 2007), Robbins (*Debugging Microsoft .NET 2.0 Applications*, Microsoft Press, 2005), Best (*Linux Debugging and Performance Tuning*, Prentice-Hall, 2005), Ford e Teorey (*Practical Debugging in C++*, Prentice-Hall, 2002), Brown (*Debugging Perl*, McGraw-Hill, 2000), e Mitchell (*Debugging Java*, McGraw-Hill, 2000) tratam da natureza especial da depuração para os ambientes citados nos seus títulos.

Uma ampla gama de fontes de informação sobre métodos de teste orientado a objeto está disponível na Internet. Uma lista atualizada das referências relevantes para as técnicas de teste pode ser encontrada no site **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**.